

一位资深工程师的 Gradle 精通指南：从基础原理到企业级自动化构建

引言：通往 Gradle 精通之路

欢迎踏上 Gradle 的学习之旅。作为一名在企业级 Java 项目中与构建系统搏斗多年的工程师，我可以告诉你，将构建自动化视为一门严肃的工程学科，而非简单的杂务，是区分优秀开发者与卓越工程师的关键分水岭。精通像 Gradle 这样强大而灵活的工具，其价值远超编译和打包代码。它是一个“力量倍增器”，能够为你和你的团队带来更快的反馈周期、更可靠的发布流程、以及更清晰、更具可维护性的项目架构。

这份指南是我为你精心设计的个人成长路线图，旨在帮助你从一名对 Gradle 仅有初步了解的 Java 开发者，成长为团队中能够独当一面的“构建大师”。我们将严格遵循官方文档的精髓，确保你学到的每一分知识都源于其设计的初衷。

这份学习路径分为五个循序渐进的阶段。每一阶段都建立在前一阶段的基础之上，逻辑清晰，层层递进。我们将从运行一个最简单的任务开始，逐步深入到多项目构建的复杂架构，最终掌握自定义插件开发和企业级性能调优的艺术。请跟随我的脚步，让我们一起揭开 Gradle 的神秘面纱，释放其全部潜能。

第一阶段：奠定基石 —— 核心概念与你的第一个构建

这一阶段的目标是为你建立一个关于 Gradle 坚实的心智模型。我们不仅要学习命令，更要深刻理解驱动 Gradle 设计的哲学，以及它与 Maven 等传统工具有何根本性的不同。

1.1 Gradle 的核心哲学：灵活性、性能与 DSL

Gradle 的设计哲学凝聚了过去几十年构建自动化领域的经验与教训，其核心在于三大支柱：一个

灵活的基于任务的模型、对性能的不懈追求，以及一种表现力极强的领域特定语言（DSL）。

- **基于任务的模型 (Task-Based Model)**：这是 Gradle 灵活性的源泉。与 Maven 固定的、线性的生命周期阶段（如 `validate`, `compile`, `test`）不同，Gradle 的核心是一个由**任务 (Task)** 构成的有向无环图 (**Directed Acyclic Graph, DAG**)¹。每个任务代表一个原子性的工作单元，比如编译代码或运行测试。你可以定义任务之间的依赖关系，Gradle 会确保它们按正确的顺序执行。这种模型允许你精确地定义构建逻辑，而不受预设阶段的束缚⁴。
- **约定优于配置 (Convention over Configuration)**：Gradle 吸收了 Maven 的这一优点，为标准项目类型（如 Java 应用）提供了合理的默认配置⁴。例如，它会默认在 `src/main/java` 目录下寻找 Java 源码。然而，与 Maven 僵化的模型不同，Gradle 的所有约定都是可以轻松覆盖和配置的，这在满足特殊构建需求时提供了巨大的便利性⁴。
- **丰富的构建脚本 (The DSL)**：Gradle 的构建脚本不仅仅是声明式的配置文件，它们是**可执行的代码**。Gradle 提供了基于 Groovy 和 Kotlin 的领域特定语言（DSL），允许你在需要时编写命令式逻辑，从而赋予你解决复杂自动化问题的终极能力⁶。在本指南中，我们将坚定地推荐并专注于现代的 **Kotlin DSL (.kts)**，因为它提供了类型安全、卓越的 IDE 支持（如代码补全和导航），极大地提升了大型项目的可维护性⁶。
- **性能即特性 (Performance as a Feature)**：从诞生之初，Gradle 就将性能视为其核心特性之一。它通过多种机制来避免不必要的工作，从而显著加快构建速度，这与在大型项目中可能变得缓慢的 Maven 形成了鲜明对比⁵。关键的性能特性包括：
 - **增量构建 (Incremental Builds)**：只重新构建自上次构建以来发生变化的部分²。
 - **构建缓存 (Build Cache)**：重用之前任何构建（甚至是你同事机器上的构建）所产生的输出²。
 - **Gradle 守护进程 (Daemon)**：在后台保持一个常驻进程，将构建信息保留在内存中，从而加速后续的构建²。

1.2 Gradle 项目剖析：关键文件解读

一个标准的 Gradle 项目由几个核心文件定义，理解它们各自的角色至关重要。

- **Gradle 包装器 (gradlew, gradlew.bat)**：这是执行任何 Gradle 构建的**唯一推荐方式**，不容商榷。它是一个随项目源码一起提交到版本控制系统的小脚本。当你运行它时，它会自动下载并使用项目在 `gradle/wrapper/gradle-wrapper.properties` 文件中定义的**特定 Gradle 版本**¹。
这种机制的价值是巨大的：它确保了项目构建的**可复现性**。无论是在你的机器、你同事的机器，还是在 CI/CD 服务器上，构建都将使用完全相同的 Gradle 版本执行，从而根除了因环境差异导致的“在我机器上可以正常工作”这类典型问题。包装器将构建环境与开发者本地安装的任何 Gradle 版本解耦，创建了一个封闭且一致的构建环境，这是迈向可靠构建的第一步，也是最关键的一步。
- **设置文件 (settings.gradle.kts)**：此文件位于项目根目录，是 Gradle 构建的入口点。它的核心职责是在多项目构建中定义项目结构，通过 `include()` 方法声明哪些子模块（或称子项

目) 需要被包含到构建中¹。此外, 它也是配置项目级仓库 (用于解析插件和依赖) 的中心位置, 确保所有模块使用统一的源¹²。

- **构建脚本 (build.gradle.kts):** 每个项目 (无论是根项目还是子项目) 都有一个自己的构建脚本。这个文件定义了该项目**如何被构建**。你将在这里应用插件、声明依赖、配置任务以及编写自定义的构建逻辑¹。

1.3 实践应用: 你的第一个 Java 项目

理论知识需要通过实践来巩固。现在, 我们将遵循官方的“初学者教程”, 从零开始创建一个标准的 Java 应用程序¹⁵。

- **分步指南:**
 1. 首先, 确保你的开发环境中已安装 JDK 8 或更高版本。
 2. 打开你的终端或命令行工具。
 3. 创建一个新的项目目录并进入: `mkdir my-first-app && cd my-first-app`。
 4. 运行 Gradle 初始化命令。我们将明确指定项目类型为 java-application 并使用 Kotlin DSL:

Shell

```
gradle init --type java-application --dsl kotlin
```

在交互式提示中, 你可以为大多数选项选择默认值¹⁶。

- **探索生成的文件:** `gradle init` 命令会为你生成一个完整的项目结构。让我们来检视一下这些文件, 并将它们与上一节的概念联系起来¹⁶:
 - `gradlew, gradlew.bat`: Gradle 包装器脚本。
 - `gradle/wrapper/`: 包含包装器的 jar 文件和配置文件。
 - `settings.gradle.kts`: 设置文件, 其中包含了 `rootProject.name = 'my-first-app'` 和 `include('app')`。
 - `app/build.gradle.kts`: app 子项目的构建脚本。
 - `app/src/main/java/` 和 `app/src/test/java/`: 遵循标准约定的源代码和测试代码目录。
- **运行你的第一个任务:** java-application 插件已经为我们预置了一些标准任务。让我们通过包装器来执行它们¹:
 - **列出所有可用任务:** `./gradlew tasks`。这将显示一个按组分类的任务列表, 包括 `build`、`run` 等²⁰。
 - **构建项目:** `./gradlew build`。这个复合任务会编译代码、运行测试, 并把应用打包成一个可执行的 JAR 文件。
 - **运行测试:** `./gradlew test`。这个任务会专门执行单元测试。
 - **运行应用:** `./gradlew run`。这个任务会执行应用的主类, 你将在控制台看到 "Hello World!" 的输出。

选择 Kotlin DSL 而非 Groovy DSL, 是一个影响深远的技术决策, 它直接关系到项目的长期可维

护性和团队的开发效率。Gradle 官方反复推荐使用 Kotlin DSL，其核心优势在于“通过语法高亮、代码补全和声明导航提供了更好的编辑体验”⁶。这直接解决了 Gradle 早期 Groovy DSL 的一个主要痛点：由于其动态特性，IDE 很难对其进行静态分析，导致在编写构建脚本时的体验远不如 Maven 的 XML²。因此，采用 Kotlin DSL 不仅仅是个人偏好或风格选择，更是对类型安全和强大工具链的一项投资。它能在编译期捕捉到更多错误，降低新成员上手项目的学习曲线，并使得重构复杂的构建逻辑变得更加安全可靠。

第二阶段：深入引擎室 —— 构建生命周期与依赖管理

在这一阶段，我们将剖析当你运行一个 Gradle 命令时，背后究竟发生了什么。理解构建生命周期是编写高效、正确构建脚本的关键。同时，我们也将精通依赖管理这一任何构建工具的核心职能。

2.1 深入理解构建生命周期

每一次 Gradle 构建都会严格遵循三个不同的阶段。你的构建脚本中的代码在哪个阶段执行，对构建的行为有着决定性的影响³。

- 三大阶段详解：

1. 初始化阶段 (Initialization):

- Gradle 在此阶段确定哪些项目参与构建。它会寻找并执行 `settings.gradle.kts` 文件。
- 根据 `settings.gradle.kts` 中的 `include()` 指令，Gradle 会为每个项目（根项目和所有子项目）创建一个 Project 对象实例²²。

2. 配置阶段 (Configuration):

- 这是构建逻辑的核心所在。Gradle 会执行每个参与构建的项目的 `build.gradle.kts` 脚本。
- 脚本中的代码（如 `plugins {}` 块、`dependencies {}` 块、任务配置等）在此阶段被评估，用于构建 Gradle 的内部模型。
- 此阶段最重要的产出是**任务图 (Task Graph)**。Gradle 会解析所有任务及其依赖关系（例如，`jar` 任务依赖于 `compileJava` 任务），并构建出一个有向无环图 (DAG)³。
- 一个至关重要的认知是：**你写在 `build.gradle.kts` 顶层的代码，是在配置阶段执行的，而不是在执行阶段。**这意味着，在配置阶段，任何任务的产出文件都不存在。

3. 执行阶段 (Execution):

- Gradle 根据你在命令行中请求的任务（例如 `./gradlew build`），以及任务图中的依赖关系，确定需要执行的任务子集。
- 然后，Gradle 按照任务图定义的顺序，执行这些任务的动作（Action）。例如，执行 `compileJava` 任务的编译动作。
- 如果一个任务的输入和输出自上次执行以来没有变化，Gradle 会跳过该任务，并在控制台将其标记为 UP-TO-DATE，这是增量构建的核心体现⁶。
- 任务图 (Task Graph, DAG):
任务图是理解 Gradle 工作模式的重中之重。Gradle 的执行模型不是线性的，而是基于依赖关系的。一个任务只有在其所有依赖的任务都成功执行完毕后，才会被执行³。这个模型是 Gradle 能够实现并行执行等高级性能特性的基础。当你请求执行一个任务时，Gradle 会反向遍历任务图，找到所有直接和间接的依赖，形成一个待执行的任务计划。

2.2 精通依赖声明

声明依赖是构建脚本最常见的任务之一。我们需要超越简单地添加库，去理解依赖的*作用域*，这对于构建清晰、高效的项目至关重要。

- api vs. implementation 的关键区别：
这是现代 Gradle 依赖管理的核心。当你使用 `java-library` 插件（用于构建库）时，它会引入这两个新的依赖配置，以精确控制依赖的可见性²⁴。
 - `implementation`：这是你应该**默认使用**的配置。它表示一个依赖是该模块的**内部实现细节**。这个依赖在模块内部的编译和运行时都是可用的，但它**不会被暴露**给消费此模块的其他模块的编译类路径²⁵。
 - `api`：此配置表示一个依赖是该模块**公共 API 的一部分**。例如，如果你的一个公共方法返回了某个库中的类型，那么这个库就必须通过 `api` 声明。这个依赖会被传递性地暴露给消费者的编译类路径²⁴。

这个区别并非无关紧要，它对多项目构建的性能有着直接且深远的影响。Gradle 的性能优势源于其增量构建能力——只重新构建发生变化的部分²。对于一个 Java 模块而言，其编译类路径是构建输入的一部分。如果类路径发生变化，该模块就必须重新编译。`api` 依赖会“泄露”到下游消费者的编译类路径中，而 `implementation` 依赖则不会²⁴。因此，当你修改一个库模块的 `implementation` 依赖时（比如升级版本），只有该库模块本身需要重新编译。然而，如果你修改的是一个 `api` 依赖，那么不仅该库模块需要重新编译，所有依赖于它的下游消费者模块也**必须**重新编译，因为它们的编译类路径也随之改变了。这揭示了一个直接的因果关系：滥用 `api` 配置会打破模块间的编译时隔离，增加构建时的耦合度，从而导致更慢、更不充分的增量构建。最佳实践是：“**默认使用 `implementation`，只有当某个依赖中的类型被你的模块的公共 API（例如，公共方法的参数、返回类型或父类）所暴露时，才将其提升为 `api` 依赖**”²⁴。

- 其他依赖配置：为了完善你的工具箱，我们还需要了解其他几个重要的配置²⁸：
 - `compileOnly`：依赖仅在编译时需要，不会被打包到最终的产出物中。适用于注解处理器或在运行时由其他环境（如 Servlet 容器）提供的 API。

- runtimeOnly：依赖在编译时不需要，但运行时必须存在。典型的例子是 JDBC 驱动或 SLF4J 的具体日志实现。
- testImplementation：测试代码专用的 implementation，用于声明单元测试和集成测试的依赖，如 JUnit、Mockito 等。

下表清晰地总结了这些关键依赖配置之间的差异：

配置 (Configuration)	描述	消费者编译类 路径	消费者运行时 类路径	主要使用场景
implementation	模块的内部实现依赖	不包含	包含	默认选择。用于隐藏实现细节，加速增量构建。
api	模块公共 API 的一部分	包含	包含	当依赖的类型出现在模块的公共接口中时使用。
compileOnly	仅编译时需要	不包含	不包含	注解处理器、编译时检查工具、运行时由环境提供的 API。
runtimeOnly	仅运行时需要	不包含	包含	日志实现、数据库驱动、插件化架构中的具体实现。
testImplementation	仅用于测试编译和运行	不适用	不适用	测试框架（JUnit, TestNG）、断言库、Mock 框架。

2.3 配置仓库

在声明依赖之前，你需要告诉 Gradle 去哪里寻找这些依赖。

- **集中式仓库管理**：现代 Gradle 的最佳实践是在根项目的 settings.gradle.kts 文件中，使用 pluginManagement 和 dependencyResolutionManagement 块来集中声明所有仓库¹²。这可以保证所有子项目都从相同的、受信任的源获取插件和依赖，避免了配置不一致的问题。

Kotlin

```
// settings.gradle.kts
pluginManagement {
    repositories {
        gradlePluginPortal()
        google()
        mavenCentral()
    }
}

dependencyResolutionManagement {
    repositories {
        google()
        mavenCentral()
    }
}
```

- **标准公共仓库**：Gradle 为最常用的公共仓库提供了便捷的简写方法，如 mavenCentral()、google()（用于 Android 开发）和 gradlePluginPortal()（用于 Gradle 插件）³⁰。
- **自定义与私有仓库**：你可以通过 maven { url = "..."} 或 ivy { url = "..."} 来声明任何自定义的 Maven 或 Ivy 兼容仓库。这对于连接公司内部的私有仓库（如 Artifactory, Nexus）至关重要。如果仓库需要认证，你可以在 credentials {...} 块中提供用户名和密码³¹。

2.4 管理传递性依赖与版本冲突

大型项目中，依赖关系网错综复杂。你的项目依赖 A，A 依赖 B，B 又依赖了 C 的 1.0 版本；同时你的项目还依赖了 D，D 依赖了 C 的 2.0 版本。这时就出现了版本冲突。

- **默认冲突解决策略**：当在依赖图中发现同一个库的多个版本时，Gradle 的默认策略是**选择最新的版本**⁵。这个“最新版本优先”的策略在大多数情况下是有效的，但也可能引入不兼容的 API 导致运行时错误。
- **分析依赖树**：要解决冲突，首先要定位它。./gradlew :app:dependencies 命令是你最有力的工具。它会为指定的项目打印出完整的依赖树，清晰地展示出每个依赖的来源以及被解析到的最终版本。当看到 (... ->...) 这样的标记时，就意味着 Gradle 在此进行了版本选择³⁸。

- **高级解决策略**：当默认策略不适用时，Gradle 提供了多种工具来精确控制版本：
 - **平台 (Platforms / BOMs)**：这是管理一组相关库版本的推荐方式。你可以导入一个“物料清单” (Bill of Materials, BOM)，它定义了一系列库的“推荐”版本。例如，Spring Boot 和 Jetpack Compose 都提供了 BOM。通过 `platform(...)` 依赖，你可以确保所有相关的库都使用经过测试、兼容的版本，而无需在每个依赖上手动指定版本号³⁵。
 - **严格版本约束 (Strictly)**：在版本声明中使用 `strictly` 可以定义一个版本范围，如果解析出的版本不在此范围内，构建将会失败。这是一种强硬的约束，用于确保不会意外地使用不兼容的版本。
 - **强制版本 (Force)**：作为最后的手段，你可以通过 `resolutionStrategy.force` `'group:name:version'` 来强制使用某个特定版本，它会覆盖依赖图中的所有其他版本。这需要非常谨慎地使用，因为它可能破坏传递性依赖的兼容性³⁹。

2.5 实践应用：使用版本目录集中管理依赖

版本目录 (Version Catalogs) 是 Gradle 7.0 引入的一项革命性功能，它彻底改变了大型项目中管理依赖的方式。

- **什么是版本目录？** 它是一种在 `libs.versions.toml` 文件中集中定义依赖坐标、版本和插件的机制，并为它们生成类型安全的访问器⁴²。

版本目录不仅仅是一个依赖列表，它是在大型多模块代码库中强制执行一致性并改善开发者体验的强大工具。它将依赖管理从一个自由格式、易于出错的字符串编辑活动，转变为一个结构化、类型安全且可发现的过程。在没有版本目录的项目中，依赖项是以“魔术字符串”的形式散布在各个 `build.gradle.kts` 文件中，这很容易导致模块间的版本不匹配和拼写错误³⁸。版本目录将这些定义集中在 `libs.versions.toml` 中⁴²。随后，Gradle 为这些定义生成类型安全的访问器（例如 `libs.androidx.core`）⁴²。这使得 IDE 能够提供自动补全和编译时检查。尝试使用一个不存在的库，如 `libs.nonexistent.library`，将直接导致构建脚本自身的编译错误，从而提供即时反馈。这种机制将依赖管理提升为构建逻辑中一流的、可验证的部分，对于企业级项目而言，这是一个在可伸缩性和可维护性方面的巨大胜利。
- **创建 `gradle/libs.versions.toml`**：我们来重构项目，将所有依赖定义移入这个新文件。它主要包含四个部分⁴²：
 - `[versions]`：定义版本号变量，便于统一升级。
 - `[libraries]`：定义库的别名和它们的 Maven 坐标 (`group:name:version`)。
 - `[bundles]`：将多个库组合成一个“包”，方便一次性添加。
 - `[plugins]`：定义插件的别名和它们的 ID 及版本。

```
Ini, TOML
# gradle/libs.versions.toml
[versions]
junit = "5.10.0"
guava = "32.1.2-jre"
```



```
[libraries]
junit-api = { group = "org.junit.jupiter", name = "junit-jupiter-api", version.ref = "junit" }
junit-engine = { group = "org.junit.jupiter", name = "junit-jupiter-engine", version.ref = "junit" }
guava = { module = "com.google.guava:guava", version.ref = "guava" }

[bundles]
testing = ["junit-api", "junit-engine"]

[plugins]
java-application = { id = "java", version = "unspecified" }
```

- **使用版本目录：**现在，更新 `app/build.gradle.kts` 文件，通过生成的类型安全访问器来引用依赖。你会立刻感受到 IDE 自动补全带来的便利⁴²。

```
Kotlin
// app/build.gradle.kts
plugins {
    // 注意：插件的引用方式略有不同
    alias(libs.plugins.java.application)
}

dependencies {
    implementation(libs.guava)
    testImplementation(libs.bundles.testing)
}
```

第三阶段：规模化扩展 —— 架构多项目构建

企业级应用很少是单体结构。这一阶段将教你如何将一个大型代码库拆分成一个清晰、可维护且性能优越的多项目构建。

3.1 模块化的原则

- **为何要模块化？** 我们将探讨模块化带来的架构优势：改善代码组织、明确团队所有权、支持并行开发，以及通过更有效的增量构建来加快构建速度¹³。一个良好模块化的项目，修改一处代码通常只需要重新构建一小部分，而不是整个系统。
- **Gradle 的模型：**一个多项目构建由一个根项目和任意数量的子项目组成。根项目负责协调整个构建，而子项目则包含具体的业务逻辑或功能模块¹³。

3.2 定义项目层次结构

- **settings.gradle.kts 的核心角色**：如前所述，这个文件是项目结构的唯一真实来源。我们将深入研究 include() 方法，学习如何添加子项目，包括嵌套的子项目¹³。

Kotlin

```
// settings.gradle.kts
rootProject.name = "my-enterprise-app"

include("core-library")
include("data-access")
include("features:user-profile")
include("features:order-processing")
```

- **项目路径 (Project Paths)**：理解基于冒号的路径表示法是与多项目构建交互的基础。例如，:core-library 指向根目录下的 core-library 子项目，而 :features:user-profile 指向 features 文件夹下的 user-profile 子项目。这个路径在声明项目间依赖时至关重要¹³。

3.3 项目间依赖

- **project() 函数**：在一个子项目的 build.gradle.kts 文件中，你可以使用 dependencies 块中的 project() 函数来声明对另一个子项目的依赖¹³。

Kotlin

```
// features/user-profile/build.gradle.kts
dependencies {
    // user-profile 模块依赖于 core-library 模块
    implementation(project(":core-library"))
    implementation(project(":data-access"))
}
```

- **工作原理**：当你声明一个项目依赖时，Gradle 会自动建立任务间的依赖关系。它会确保被依赖的项目（如 :core-library）在依赖它的项目（如 :features:user-profile）之前被构建。然后，它会将 :core-library 的输出（即编译后的类文件和资源）添加到 :features:user-profile 的编译和运行时类路径中¹³。

3.4 共享构建逻辑（第一部分）：buildSrc 目录

随着多项目构建的规模扩大，你会发现不同子项目的 `build.gradle.kts` 文件中开始出现大量重复的配置，例如应用相同的插件、配置相同的 Java 版本、添加相同的通用依赖等。

- **buildSrc 的引入**：为了解决配置重复的问题，Gradle 提供了一个名为 `buildSrc` 的特殊目录，它必须位于项目的根目录下。你可以将所有共享的构建逻辑放入此目录¹⁴。
- **Gradle 如何处理 buildSrc**：Gradle 对 `buildSrc` 目录有特殊的处理机制。它会将其视为一个独立的、内嵌的 Gradle 项目。在评估主构建的任何脚本之前，Gradle 会首先自动编译 `buildSrc` 目录中的代码，并将其编译产物添加到主构建中所有 `build.gradle.kts` 文件的类路径上。这实际上等同于为你的构建创建了一个私有的、自动应用的插件⁴⁹。
- **典型用例**：`buildSrc` 是集中管理构建逻辑的绝佳起点。你可以用它来：
 - 在 `buildSrc/src/main/kotlin` 中定义 Kotlin 对象来统一管理依赖版本（这是版本目录出现之前的常用做法）。
 - 编写自定义任务类。
 - 定义“约定插件”（我们将在第四阶段深入探讨）⁴⁹。

尽管 `buildSrc` 非常方便，但它也像一把双刃剑。虽然它能有效地集中管理逻辑，但也可能成为性能瓶颈。Gradle 的工作机制决定了，在评估任何其他构建脚本之前，它会先编译 `buildSrc`⁴⁹。这意味着，对 `buildSrc` 内部代码的任何微小改动，都会导致 `buildSrc` 本身被重新编译，并且会使整个项目的配置缓存失效（如果已启用）。在大型项目中，这可能会给每次修改构建逻辑后的构建增加数秒甚至更长的时间。因此，`buildSrc` 虽然是共享逻辑的优秀起点，但它创建了一个单体的构建逻辑块。这引出了更高级的概念，如“复合构建”和独立插件（将在后续阶段介绍），它们将构建逻辑分解为独立的、可单独缓存的单元——就像我们对应用程序代码进行模块化一样。将 `buildSrc` 视为通往企业级构建逻辑管理的垫脚石，而非最终目的地。

3.5 实践应用：模块化你的应用

现在，我们将把第一阶段创建的单项目应用重构成一个结构更清晰的多项目构建。

1. **创建新结构**：
 - 在项目根目录下，创建一个新的 `core` 目录。
 - 将 `app/src/main/java` 中的部分通用逻辑代码移动到 `core/src/main/java` 中。
 - 为 `core` 模块创建一个 `core/build.gradle.kts` 文件。
2. **更新配置**：
 - 修改根目录的 `settings.gradle.kts`，确保它包含了新的 `core` 模块：

```
Kotlin
include("app", "core")
```
 - 在 `app/build.gradle.kts` 中，添加对 `core` 模块的依赖：

```
Kotlin
dependencies {
    implementation(project(":core"))
    //... 其他依赖
}
```

- 在 core/build.gradle.kts 中，应用 java-library 插件，因为它是一个库模块。

3. 集中化逻辑：

- 创建 buildSrc 目录，并在其中创建 buildSrc/build.gradle.kts 以启用 Kotlin DSL 支持。
- 在 buildSrc/src/main/kotlin 中创建一个 Kotlin 文件，例如 Dependencies.kt，用于定义共享的依赖版本。
- 将之前在 app/build.gradle.kts 中的通用配置（例如 Java 版本配置）提取出来，为后续创建约定插件做准备。

第四阶段：自动化的艺术 —— 自定义任务与约定插件

在这一阶段，你将从 Gradle 的使用者转变为真正的自动化专家。你将学会如何通过编写自己的逻辑来扩展 Gradle，以满足项目独特的自动化需求。

4.1 编写自定义任务

Gradle 所做的一切都是通过任务来完成的。现在，我们将学习如何创建自己的任务。

- **实现任务类：**最佳实践是创建一个继承自 org.gradle.api.DefaultTask 的自定义类²¹。将任务逻辑封装在类中，可以使其更具可测试性和可重用性。通常，这些类会放在 buildSrc/src/main/kotlin 目录下。
- **@TaskAction 注解：**在你的任务类中，使用 @TaskAction 注解来标记那个包含了任务核心执行逻辑的方法。当 Gradle 执行该任务时，这个被注解的方法就会被调用⁵¹。
- **任务输入与输出：**这是实现高性能构建的关键。通过明确声明任务的输入（文件、属性）和输出（文件、目录），你就为 Gradle 的增量构建功能提供了必要的信息。如果任务的输入自上次执行以来没有变化，Gradle 就可以安全地跳过这个任务的执行，并将其标记为 UP-TO-DATE，从而节省大量时间⁵⁴。
 - **注解：**使用 @Input、@InputFile、@InputDirectory 来标记输入属性；使用 @OutputFile、@OutputDirectory 来标记输出属性。
 - **惰性属性 (Lazy Properties)：**为了实现更高效和更安全的配置，应始终使用 Gradle 的属性类型（如 Property<T>、RegularFileProperty、DirectoryProperty）来声明输入和

输出。这允许任务的配置被推迟到真正需要时才进行评估。

```
Kotlin
// 在 buildSrc/src/main/kotlin/com/example/GenerateVersionFileTask.kt
package com.example

import org.gradle.api.DefaultTask
import org.gradle.api.file.RegularFileProperty
import org.gradle.api.provider.Property
import org.gradle.api.tasks.Input
import org.gradle.api.tasks.OutputFile
import org.gradle.api.tasks.TaskAction

abstract class GenerateVersionFileTask : DefaultTask() {

    @get:Input
    abstract val appVersion: Property<String>

    @get:OutputFile
    abstract val outputFile: RegularFileProperty

    @TaskAction
    fun generate() {
        outputFile.get().asFile.writeText("version=${appVersion.get()}")
    }
}
```

4.2 共享构建逻辑（第二部分）：约定插件

虽然 buildSrc 可以集中代码，但现代 Gradle 推荐的、更结构化的共享逻辑方式是**约定插件 (Convention Plugins)**。这些插件封装了针对某一类项目的“约定”或“标准配置”，例如“这是一个 Spring Boot 库”或“这是一个 Android 特性模块”⁴⁹。

- **实现方式**：约定插件通常以**预编译脚本插件**的形式实现，即在 buildSrc/src/main/kotlin 目录下创建 .gradle.kts 文件。该文件的文件名（不含扩展名）将自动成为插件的 ID⁵⁷。例如，my-java-library-conventions.gradle.kts 文件会创建一个 ID 为 my-java-library-conventions 的插件。
- **核心优势**：约定插件是类型安全的、易于测试的，并且能让你用一行代码就应用一整套复杂的配置（包括其他插件、依赖、任务配置等）。这极大地简化了子项目的构建脚本，使其专注于声明性的内容，而非重复的样板代码⁴⁹。

```
Kotlin
// 在 buildSrc/src/main/kotlin/my-java-library-conventions.gradle.kts
```

```

// 应用核心插件
plugins {
    `java-library`
}

// 配置仓库
repositories {
    mavenCentral()
}

// 添加通用依赖
dependencies {
    testImplementation("org.junit.jupiter:junit-jupiter-api:5.10.0")
    testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine")
}

// 配置 Java 工具链
java {
    toolchain {
        languageVersion.set(JavaLanguageVersion.of(17))
    }
}

// 配置测试任务
tasks.withType<Test>().configureEach {
    useJUnitPlatform()
}

```

4.3 实践应用：创建你的第一个约定插件

现在，我们将把 buildSrc 中的零散逻辑重构为一个结构化的约定插件。

1. **创建插件文件：**在 buildSrc/src/main/kotlin 目录下创建 java-library-conventions.gradle.kts 文件，并将上一节中的代码复制进去。
2. **应用插件：**修改 core 模块的 core/build.gradle.kts 文件。删除所有重复的配置，只保留应用插件和模块特有的依赖。

Kotlin

```

// core/build.gradle.kts
plugins {

```



```

    id("java-library-conventions")
}

dependencies {
    // 这里只保留 core 模块特有的依赖
}

```

3. 注册并使用自定义任务：

- 在 java-library-conventions.gradle.kts 中注册我们之前创建的 GenerateVersionFileTask。

Kotlin

// 在 java-library-conventions.gradle.kts 文件末尾添加

```

tasks.register<com.example.GenerateVersionFileTask>("generateVersionFile") {
    appVersion.set(project.version.toString())
    outputFile.set(project.layout.buildDirectory.file("gen/version.properties"))
}

```

- 现在，你可以在任何应用了此插件的模块中运行 `./gradlew generateVersionFile`。

下表对比了创建自定义插件的不同策略，帮助你根据场景选择最合适的方法。

策略	实现位置	主要用例	优点	缺点
脚本插件	单独的 .gradle.kts 文件	单个项目内的简单逻辑共享	简单快捷，无需额外设置	不推荐 。可维护性差，难以测试，跨项目复用困难 ⁵⁷ 。
预编译脚本插件	buildSrc 目录或复合构建	约定插件 。在单个代码仓库内的多项目间共享构建逻辑。	自动编译和类路径可用，类型安全，易于测试，IDE 支持良好 ⁴⁹ 。	任何改动都会导致 buildSrc 重新编译，可能影响构建性能 ⁴⁹ 。
二进制/独立插件	独立的 Gradle 项目	共享插件 。需要跨多个不同代码仓库或与社区共享的通用插件。	最大化的可重用性和封装性，可独立版本化和发布 ⁵⁷ 。	设置更复杂，需要发布到仓库才能使用，开发调试周期稍长。

第五阶段：登峰造极 —— 高级插件开发与性能调优

这是通往专家之路的最后冲刺。我们将学习如何创建可配置、可分发的插件，并掌握分析和优化构建性能的终极工具。

5.1 高级插件开发

- **创建插件扩展 (Extensions)**：为了让你的插件更加灵活，你需要为用户提供配置它的能力。通过创建一个“扩展”类，你可以让用户在他们的 `build.gradle.kts` 中使用一个自定义的 DSL 块来配置插件的行为⁶⁰。

Kotlin

// 在 buildSrc 中定义扩展类

```
abstract class MyPluginExtension {  
    abstract val message: Property<String>  
    init {  
        message.convention("Default Message")  
    }  
}
```

// 在插件的 apply 方法中注册扩展

```
val extension = project.extensions.create("myPluginConfig", MyPluginExtension::class.java)
```

// 用户可以在 build.gradle.kts 中这样配置

```
myPluginConfig {  
    message.set("Hello from my plugin!")  
}
```

- **独立插件项目**：为了在多个代码仓库或团队之间共享插件，最佳实践是将其开发为一个完全独立的 Gradle 项目⁵⁷。这个项目本身就是一个标准的 Gradle 项目，它使用 `java-gradle-plugin`，最终产物是一个可以发布到 Maven 仓库的 JAR 文件。
- **发布到 Gradle 插件门户**：将你的插件分享给全世界！我们将完整地走一遍将独立插件发布到 Gradle 官方插件门户的流程⁶⁴。
 1. **账户设置**：在 plugins.gradle.org 注册账户并获取 API 密钥⁶⁴。
 2. **配置 plugin-publish 插件**：在你的插件项目的构建脚本中，应用并配置 `com.gradle.plugin-publish` 插件。
 3. **定义插件元数据**：在 `gradlePlugin { ... }` 块中，详细定义插件的 ID、显示名称、描述、标签、源码仓库地址等信息⁶⁴。
 4. **执行发布任务**：运行 `./gradlew publishPlugins` 命令。首次发布需要经过人工审核，后续

更新会自动发布⁶⁴。

5.2 性能优化工具箱：多层次优化策略

Gradle 的性能不是单一功能，而是一系列协同工作的优化层。理解这个层次结构是有效提升构建速度的关键。

优化层	缓存内容	何时生效	如何启用	关键考量
增量构建	任务的输入/输出状态	在单个工作区内，避免重新执行未变化的 UP-TO-DATE 任务。	默认启用。开发者需正确注解任务的输入输出。	任务实现必须正确、完整地声明所有输入和输出。
构建缓存	任务的输出文件	跨工作区、跨机器重用任务产出 (FROM-CACHE)，即使执行 clean 后依然有效。	org.gradle.caching=true	任务必须是 确定性的 （相同的输入总是产生相同的输出）。
配置缓存	整个配置阶段的结果（任务图）	在构建脚本和环境未变时，完全跳过配置阶段，直接进入执行阶段。	org.gradle.configuration-cache=true	构建逻辑在配置阶段不能读取外部状态（如环境变量、文件内容）。

- **第一层：构建缓存 (Build Cache)**

- **概念：**构建缓存通过存储可重定位的任务输出来避免重复工作。当 Gradle 准备执行一个任务时，它会根据任务的所有输入（包括源文件、类路径、编译器版本等）计算一个唯一的缓存键。如果这个键在缓存中存在，Gradle 会直接从缓存中拉取输出，而不是执行任务²⁷。
- **本地缓存 vs. 远程缓存：**
 - **本地缓存**默认位于 `~/.gradle/caches` 目录下，它能重用你自己在本地执行过的任务输出。
 - **远程缓存**通常是一个共享的 HTTP 服务器，它允许整个团队（包括 CI 服务器）共享

和重用任务输出，极大地提升了团队整体的构建效率²⁷。

- **配置：**通过在 `gradle.properties` 中设置 `org.gradle.caching=true` 来启用本地缓存。远程缓存则需要在 `settings.gradle.kts` 中进行配置⁶⁶。

Kotlin

```
// settings.gradle.kts
buildCache {
    local {
        isEnabled = true
    }
    remote<HttpBuildCache> {
        url = uri("https://my-company.com/gradle-cache")
        isPush = true // CI 服务器通常设置为 true
        credentials {
            username = "user"
            password = "password"
        }
    }
}
```

- **第二层：配置缓存 (Configuration Cache)**

- **概念：**这是 Gradle 近年来最重要的性能改进之一。对于大型多项目构建，配置阶段本身就可能耗费大量时间。配置缓存会将整个配置阶段的结果——即构建好的任务图——序列化并存储起来。在后续构建中，如果构建脚本、`gradle.properties` 等配置输入没有变化，Gradle 就可以完全跳过配置阶段，直接加载缓存的任务图并进入执行阶段，从而实现秒级启动⁶⁹。
- **启用：**通过命令行参数 `--configuration-cache` 或在 `gradle.properties` 中设置 `org.gradle.configuration-cache=true` 来启用⁷⁰。
- **缓存失效：**理解什么会使配置缓存失效至关重要。任何在配置阶段读取的、且未被 Gradle 跟踪的外部状态（例如，直接读取环境变量、文件系统）的改变，都会导致缓存失效⁶⁹。编写与配置缓存兼容的构建逻辑是高级 Gradle 开发的一项核心技能。

- **第三层：使用构建扫描和 Gradle Profiler 进行深度分析**

- **构建扫描 (Build Scans)：**通过在命令后添加 `--scan` 参数，你可以为一次构建生成一个可分享的、基于 Web 的详细报告⁷²。这个报告是诊断性能问题和构建失败的终极工具。你可以在“Performance”标签页中看到构建各个阶段的耗时、任务执行时间线、依赖解析详情等，从而精确定位瓶颈⁷³。
- **Gradle Profiler：**这是一个用于对 Gradle 构建进行基准测试和性能剖析的专用工具⁷⁵。它允许你编写场景 (Scenarios)，例如“无操作构建”、“修改一个私有方法”、“修改一个公共 API”，然后反复运行这些场景以收集统计上显著的性能数据。它还可以与 JProfiler、Async Profiler 等专业的 Java 剖析工具集成，生成火焰图，帮助你深入分析构建逻辑中耗时的代码热点。

5.3 实践应用：性能调优实战

1. **启用缓存**：在你的项目的 `gradle.properties` 文件中，添加以下行：
`Properties`
`org.gradle.caching=true`
`org.gradle.configuration-cache=true`
2. **首次运行**：执行 `./gradlew build`。这次构建会填充缓存，可能会稍慢。
3. **二次运行**：再次执行 `./gradlew build`。你会看到大量任务被标记为 `FROM-CACHE` 或 `UP-TO-DATE`，并且构建顶端会显示 `Reusing configuration cache.`，整个构建时间将大幅缩短。
4. **分析构建扫描**：执行 `./gradlew build --scan`。打开生成的链接，花时间探索“Performance”标签页。查看“Configuration”耗时和“Task execution”时间线，找出最耗时的任务。
5. **使用 Profiler**：安装并使用 Gradle Profiler 来对一个常见场景进行基准测试。例如，创建一个场景文件，模拟修改一个 `core` 模块中的内部实现类，然后运行基准测试，观察 `app` 模块是否需要重新编译。这将直观地验证 `implementation` 依赖所带来的性能优势。

结论：你的 Gradle 专家之旅

恭喜你，至此你已经完成了一次从 Gradle 新手到专家的系统性学习。我们回顾一下你所达成的里程碑：你从理解 Gradle 以任务为核心、追求性能的哲学开始；掌握了构建生命周期和现代化的依赖管理，特别是 `api` 与 `implementation` 的深刻区别以及版本目录的强大功能；你学会了如何使用多项目构建来架构可伸缩的企业级应用，并利用 `buildSrc` 和约定插件来共享和规范构建逻辑；最后，你迈入了高级领域，学会了编写可配置的自定义插件，并掌握了构建缓存、配置缓存、构建扫描和 Gradle Profiler 这一整套性能优化工具。

你的旅程并未就此结束。Gradle 和它的生态系统在不断发展。要保持你的专家地位，请保持对 Gradle 官方博客和发布说明的关注，尝试为社区中的开源插件贡献代码，更重要的是，将在本指南中学到的原则和技术应用到你的日常工作中。去发现你项目中那些缓慢、脆弱或重复的构建逻辑，并用你新掌握的知识去重构它们。

你现在所拥有的，不仅仅是操作一个工具的技能，而是一种构建自动化的思维方式。你已经具备了成为任何团队中构建领域专家的能力，能够通过构建系统的优化，为整个开发流程带来切实的效率提升和质量保障。祝你在未来的工程实践中，运用自如，不断精进。

引用的著作

1. Core Concepts - Gradle User Manual, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/gradle_basics.html
2. Maven vs Gradle, which is right for you? - Buildkite, 访问时间为 十月 17, 2025,
<https://buildkite.com/resources/comparison/maven-vs-gradle/>
3. Gradle Build Lifecycle - Gradle User Manual, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/build_lifecycle_intermediate.html
4. Gradle and Maven Comparison - Gradle, 访问时间为 十月 17, 2025,
<https://gradle.org/maven-and-gradle/>
5. Migrating Builds From Apache Maven - Gradle User Manual, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/migrating_from_maven.html
6. Gradle build overview | Android Studio, 访问时间为 十月 17, 2025,
<https://developer.android.com/build/gradle-build-overview>
7. Gradle - Wikipedia, 访问时间为 十月 17, 2025,
<https://en.wikipedia.org/wiki/Gradle>
8. Gradle - Inedo Documentation, 访问时间为 十月 17, 2025,
<https://docs.inedo.com/docs/buildmaster-platforms-java-gradle>
9. Migrate your build configuration from Groovy to Kotlin | Android Studio, 访问时间为 十月 17, 2025,
<https://developer.android.com/build/migrate-to-kotlin-dsl>
10. Gradle Kotlin DSL Primer, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/kotlin_dsl.html
11. Maven vs. Gradle: A Detailed Comparison of Pros and Cons with Examples - Medium, 访问时间为 十月 17, 2025,
<https://medium.com/@ahmettemelkundupoglu/maven-vs-gradle-a-detailed-comparison-of-pros-and-cons-with-examples-594ba33cc57f>
12. Configure your build | Android Studio, 访问时间为 十月 17, 2025,
<https://developer.android.com/build>
13. Multi-Project Builds - Gradle User Manual, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/multi_project_builds.html
14. Structuring and Organizing Gradle Projects, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/organizing_gradle_projects.html
15. Gradle User Manual, 访问时间为 十月 17, 2025,
<https://docs.gradle.org/current/userguide/userguide.html>
16. Getting Started - Gradle User Manual, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/getting_started_eng.html
17. Understanding Gradle — An Introduction for Java Developers - Medium, 访问时间为 十月 17, 2025,
<https://medium.com/@AlexanderObregon/mastering-gradle-an-introduction-for-java-developers-d686dbc026ee>
18. Getting Started with Gradle | IntelliJ IDEA Documentation - JetBrains, 访问时间为 十月 17, 2025,
<https://www.jetbrains.com/help/idea/getting-started-with-gradle.html>
19. Getting Started with JavaFX, 访问时间为 十月 17, 2025,
<https://openjfx.io/openjfx-docs/>
20. Gradle Tutorial - Tasks Overview - YouTube, 访问时间为 十月 17, 2025,
<https://www.youtube.com/watch?v=FB5muea-2Z0>

21. Understanding Tasks - Gradle User Manual, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/more_about_tasks.html
22. Build Lifecycle - Gradle User Manual, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/build_lifecycle.html
23. Chapter 20. The Build Lifecycle - Huihoo, 访问时间为 十月 17, 2025,
https://docs.huihoo.com/gradle/2.13/userguide/build_lifecycle.html
24. The Java Library Plugin - Gradle User Manual - Huihoo, 访问时间为 十月 17, 2025,
https://docs.huihoo.com/gradle/4.7/userguide/java_library_plugin.html
25. The Java Library Plugin - Gradle User Manual, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/java_library_plugin.html
26. Difference between implementation, API ,compile and runtimeOnly in Gradle
Dependency, 访问时间为 十月 17, 2025,
<https://abhiappmobiledeveloper.medium.com/difference-between-implementation-api-compile-and-runtimeonly-in-gradle-dependency-55b70215d245>
27. Build Cache - Gradle User Manual - Huihoo, 访问时间为 十月 17, 2025,
https://docs.huihoo.com/gradle/4.7/userguide/build_cache.html
28. Add build dependencies | Android Studio, 访问时间为 十月 17, 2025,
<https://developer.android.com/build/dependencies>
29. Gradle Dependency Management. From Chaos to Order: The Key to Efficient
Project Development | by Roman Glushach, 访问时间为 十月 17, 2025,
<https://romanglushach.medium.com/gradle-dependency-management-from-chaos-to-order-the-key-to-efficient-project-development-8ea0df9dd301>
30. Declaring Repositories Basics - Gradle User Manual, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/declaring_repositories_basics.html
31. 3. Declaring repositories - Gradle User Manual, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/declaring_repositories.html
32. Working with the Gradle registry - GitHub Docs, 访问时间为 十月 17, 2025,
<https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-gradle-registry>
33. Gradle Repository - Cloudsmith.io, 访问时间为 十月 17, 2025,
<https://help.cloudsmith.io/docs/gradle-repository>
34. Using Repository with Gradle - What is Repsy?, 访问时间为 十月 17, 2025,
<https://docs.repsy.io/maven/using-repository-with-gradle/>
35. Gradle dependency resolution | Android Studio | Android Developers, 访问时间为
十月 17, 2025,
<https://developer.android.com/build/gradle-dependency-resolution>
36. How do I resolve dependency conflicts with Gradle? - Stack Overflow, 访问时间
为 十月 17, 2025,
<https://stackoverflow.com/questions/27004899/how-do-i-resolve-dependency-conflicts-with-gradle>
37. Manage Resolution Conflicts - MicroEJ Documentation, 访问时间为 十月 17, 2025,
<https://docs.microej.com/en/latest/SDK6UserGuide/manageResolutionConflicts.html>
38. 1. Declaring dependencies - Gradle User Manual, 访问时间为 十月 17, 2025,

- https://docs.gradle.org/current/userguide/declaring_dependencies.html
39. Resolve dependency conflict in Gradle - Vijay Vankhede, 访问时间为 十月 17, 2025, <https://lucid-mestorf-7bb5ab.netlify.app/gradle-dependency-conflict/>
 40. Platforms - Gradle User Manual, 访问时间为 十月 17, 2025, <https://docs.gradle.org/current/userguide/platforms.html>
 41. Using Resolution Rules - Gradle User Manual, 访问时间为 十月 17, 2025, https://docs.gradle.org/current/userguide/resolution_rules.html
 42. Version Catalogs - Gradle User Manual, 访问时间为 十月 17, 2025, https://docs.gradle.org/current/userguide/version_catalogs.html
 43. RefreshVersions ❤ Gradle Version Catalog - GitHub, 访问时间为 十月 17, 2025, <https://github.com/Splitties/refreshVersions/wiki/RefreshVersions-%E2%99%A5%E%F%B8%8F-Gradle-Version-Catalog>
 44. Migrate your build to version catalogs | Android Studio, 访问时间为 十月 17, 2025, <https://developer.android.com/build/migrate-to-catalogs>
 45. Gradle Version Catalog - Ryan W - Medium, 访问时间为 十月 17, 2025, <https://callmeryan.medium.com/gradle-version-catalog-728111fa210f>
 46. Structuring Multi-Project Builds - Gradle User Manual, 访问时间为 十月 17, 2025, https://docs.gradle.org/current/userguide/multi_project_builds_intermediate.html
 47. Getting Started | Creating a Multi Module Project - Spring, 访问时间为 十月 17, 2025, <https://spring.io/guides/gs/multi-module/>
 48. Gradle Multi Project Build Guide | Medium, 访问时间为 十月 17, 2025, <https://medium.com/@AlexanderObregon/gradle-multi-project-builds-organizing-and-scaling-your-projects-766e911ac88b>
 49. Sharing Build Logic using buildSrc - Gradle User Manual, 访问时间为 十月 17, 2025, https://docs.gradle.org/current/userguide/sharing_build_logic_between_subprojects.html
 50. Organizing Build Logic - Gradle User Manual - Huihoo, 访问时间为 十月 17, 2025, https://docs.huihoo.com/gradle/4.7/userguide/organizing_build_logic.html
 51. Gradle Custom Task | Baeldung, 访问时间为 十月 17, 2025, <https://www.baeldung.com/gradle-custom-task>
 52. Chapter 58. Writing Custom Task Classes, 访问时间为 十月 17, 2025, http://sorcersoft.org/project/site/gradle/userguide/custom_tasks.html
 53. Custom Gradle Tasks with Kotlin - Medium, 访问时间为 十月 17, 2025, <https://medium.com/android-news/custom-gradle-tasks-with-kotlin-e0f8659628a6>
 54. Advanced Tasks - Gradle User Manual, 访问时间为 十月 17, 2025, https://docs.gradle.org/current/userguide/custom_tasks.html
 55. Gradle Task Inputs and Outputs - Tom Gregory, 访问时间为 十月 17, 2025, <https://tomgregory.com/gradle/gradle-task-inputs-and-outputs/>
 56. Gradle - Understanding Build Caching - Martin Vysny, 访问时间为 十月 17, 2025, <https://mvysny.github.io/gradle-build-cache/>
 57. Understanding Plugins - Gradle User Manual, 访问时间为 十月 17, 2025, <https://docs.gradle.org/current/userguide/plugins.html>
 58. Sharing build logic between subprojects Sample - Gradle User Manual, 访问时间

为 十月 17, 2025,

https://docs.gradle.org/current/samples/sample_convention_plugins.html

59. Gradle Plugins and Composite Builds - Nicola Corti, 访问时间为 十月 17, 2025,
<https://ncorti.com/blog/gradle-plugins-and-composite-builds>
60. Write Gradle plugins | Android Studio | Android Developers, 访问时间为 十月 17, 2025,
<https://developer.android.com/build/extend-agp>
61. Gradle Plugin Configuration using Extensions - Contentsquare Engineering Blog, 访问时间为 十月 17, 2025,
<https://engineering.contentsquare.com/2024/build-configuration-with-extensions-in-gradle-plugins/>
62. Standalone Gradle Plugin in Gradle DSL | by James Shepherd - Medium, 访问时间为 十月 17, 2025,
<https://medium.com/@jamesashepherd/standalone-gradle-plugin-in-gradle-dsl-19585ccbdb91>
63. Custom Gradle Plugin as Standalone project - YouTube, 访问时间为 十月 17, 2025,
<https://www.youtube.com/watch?v=CbHslSLsAWI>
64. Publishing Plugins to the Gradle Plugin Portal - Gradle User Manual, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/publishing_gradle_plugins.html
65. How do I publish my plugin to the Plugin Portal using version 1.0+?, 访问时间为 十月 17, 2025,
<https://plugins.gradle.org/docs/publish-plugin>
66. Build Cache - Gradle User Manual, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/build_cache.html
67. Gradle | Cache | Depot Documentation, 访问时间为 十月 17, 2025,
<https://depot.dev/docs/cache/reference/gradle>
68. Enable Gradle build cache | OpenRewrite Docs, 访问时间为 十月 17, 2025,
<https://docs.openrewrite.org/recipes/gradle/enablegradlebuildcache>
69. Configuration Cache - Gradle User Manual, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/configuration_cache.html
70. Enabling the Configuration Cache - Gradle User Manual, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/configuration_cache_enabling.html
71. Gradle Config Cache Reuse on CI - Jason Pearson, 访问时间为 十月 17, 2025,
<https://www.jasonpearson.dev/gradle-config-cache-reuse-on-ci/>
72. Build Scan Basics - Gradle User Manual, 访问时间为 十月 17, 2025,
https://docs.gradle.org/current/userguide/build_scans.html
73. Build Scan - Gradle User Manual, 访问时间为 十月 17, 2025,
<https://docs.gradle.org/current/userguide/inspect.html>
74. Run a free Build Scan® for Gradle, Apache Maven, and sbt builds, 访问时间为 十月 17, 2025,
<https://gradle.com/scans/gradle/>
75. gradle/gradle-profiler: A tool for gathering profiling and ... - GitHub, 访问时间为 十月 17, 2025,
<https://github.com/gradle/gradle-profiler>
76. Gradle Profiler, 访问时间为 十月 17, 2025,
<https://community.gradle.org/gradle-profiler/>
77. The Gradle Profiler: Part 1 Introduction - Steemit, 访问时间为 十月 17, 2025,
<https://steemit.com/android/@areitz/the-gradle-profiler-part-1-introduction>

78. Benchmarking builds with Gradle-Profiler - DEV Community, 访问时间为 十月 17, 2025,
<https://dev.to/autonomousapps/benchmarking-builds-with-gradle-profiler-0a8>